Untitled 16

Module: Web Application Security **Topic:** SQL Injection (SQLi)
Lab: TryHackMe — SQL Injection Lab
Instructor: Tinashe Matanda, Cybersecurity Instructor

---

1. What you're doing and why

In our last session we cracked open SQL Injection together. We didn't finish the room, and that was on purpose — the rest of it is your job. This assignment has **two parts**:

1. **Finish the TryHackMe room** end to end. Every task, every flag.
2. **Write a journal / write-up** documenting what you did, how you did it, and what you learnt.

The lab is the easy part. The journal is the part that will actually grow you.

Why a journal? (Read this — don't skip it.)

In the real world, when you find a vulnerability, nobody pays you for the finding itself. They pay you for **the report you write about the finding**. A bug bounty hunter who can pop a SQLi but can't write it up clearly gets paid pennies. The one who writes a clean, reproducible, well-explained report gets paid thousands of dollars for the same bug. Same vulnerability. Different write-up. Different cheque.

Reports and write-ups are how cybersecurity professionals communicate value. Whether it's:

- A **penetration test report** to a client,
- A **bug bounty submission** to a triage team,
- An **incident report** to your CISO after a breach,
- Or a **threat intel article** on LinkedIn,

…it's all the same skill. You learn it now, on a free TryHackMe room, while the stakes are zero. By the time the stakes are R50 000 a report, this must be muscle memory.

A journal is the **training-wheels version** of a professional report. It's less formal, more reflective, and it lets you build the habit of documenting your thinking *as you work*, not after.

What we're looking for in a good journal

A good journal is **honest**, **structured**, and **technical**. That means:

- It explains what you did and *why*, not just commands you copied.
- It admits where you got stuck. Saying *"I tried X, got error Y, then realised Z"* is gold. It shows critical thinking.
- It includes evidence — screenshots of payloads, responses, flag captures.
- It rewords concepts in your own voice. If your write-up reads like the TryHackMe task description with synonyms, you've missed the point.

---

2. Quick revision — what we covered in class

Before you touch the lab, make sure these concepts are sitting comfortably in your head. I am **not** going to repeat the lab examples here — that would be doing your thinking for you. Instead, I'll re-explain the same ideas using **completely different scenarios**. If you can map them back onto what we did in class, you've understood the concept. If you can't, go back and re-watch the recording before starting the lab.

2.1 What SQL injection actually is

SQLi happens when a web application takes user input and shoves it directly into a SQL query without checking what's in it. The application *trusts* that whatever you typed is innocent. You exploit that trust by typing things that aren't innocent — payloads that change the meaning of the query.

Different scenario to think about: Imagine a school's online library system. To check who borrowed a book, the librarian's web tool runs:

```
SELECT borrower_name FROM loans WHERE book_isbn = '<whatever you type>';
```

A normal user types: `9780132350884` The query becomes: `SELECT borrower_name FROM loans WHERE book_isbn = '9780132350884';` Fine. Returns one borrower.

You type: `9780132350884' OR '1'='1` The query becomes: `SELECT borrower_name FROM loans WHERE book_isbn = '9780132350884' OR '1'='1';`

Now the database returns **every borrower in the loans table**, because `'1'='1'` is always true. You've never seen the lab's articles page in your life and you can already see why this is dangerous.

2.2 The payload mindset

A **payload** is what you type instead of what a normal user would type. The whole game of SQLi is figuring out *what to type* so that the back-end SQL query gets rewritten in a way that benefits

you.

Three things you must always be doing in your head:

1. **Guess the underlying query.** What is this app probably running on the back-end?
2. **Find the breakage point.** Where in that query does my input land?
3. **Craft the payload.** What can I type so the query still parses, but does what *I* want?

This is critical thinking. You have a problem with it, it will have a problem with you.

2.3 The semicolon and the comment

- `;` ends a SQL statement.
- `--` (two dashes) tells SQL "ignore everything after this on this line."

That comment trick is your best friend. It lets you cut off whatever the developer wrote *after* your input — `LIMIT 1`, closing quotes, `AND status = 'public'`, all of it — and replace it with nothing.

2.4 AND vs OR — the store-run logic

Remember the store-run analogy? Let me give you a fresh one.

Imagine your boss says: *"You can leave early if you finished your report **AND** your timesheet is in."* Both must be true. Miss one, you're staying.

Now imagine she says: *"You can leave early if you finished your report **OR** it's your birthday."* Either one. Doesn't matter that you didn't finish — if it's your birthday, you're out.

This is exactly why authentication bypass works. The login query checks `username = 'X' AND password = 'Y'` — both must match. You inject an `OR 1=1` and now the database thinks: *"either both match, or one equals one… well, one does equal one, so let them in."*

2.5 UNION — pulling from a table you're not supposed to touch

`UNION` glues two `SELECT` queries together into one result set. It only works if **both selects return the same number of columns**. That's the only rule that matters for our purposes.

Different scenario: A property listings website lets you search rentals by suburb. The URL is `https://example.com/rentals?suburb=Sandton`. Under the hood:

```
SELECT title, price, bedrooms FROM listings WHERE suburb = 'Sandton';
```

Three columns. Now suppose the site also has an internal `agents` table with `agent_name`, `agent_email`, `agent_commission_rate` that you're absolutely not supposed to see. If the

suburb parameter is unsanitised, you can do:

```
Sandton' UNION SELECT agent_name, agent_email, agent_commission_rate FROM
agents--
```

Same number of columns on both sides? Yes — three each. The UNION succeeds. The page now displays agent emails and commission rates where it used to display rentals. You've **spidered out** of the listings table into a completely different one. That's the whole UNION-based SQLi technique in one go.

2.6 Finding the column count (the mock-value technique)

You don't usually know how many columns the original query is pulling. So you guess, using throwaway numbers:

```
' UNION SELECT 1--
' UNION SELECT 1,2--
' UNION SELECT 1,2,3--
' UNION SELECT 1,2,3,4--
```

You keep going until the page stops throwing an error. The number of columns where the error disappears = the number of columns the original query was selecting. Now you know how to structure the right side of your UNION.

2.7 information_schema — the database snitch

Every MySQL/Postgres install has a built-in database called `information_schema`. It exists so the DBMS can keep track of itself. You can't delete it. It snitches on the entire structure of every database on the server.

Two queries you must know cold:

- `SELECT table_name FROM information_schema.tables WHERE table_schema = 'database_name'` → gives you all tables in a database.
- `SELECT column_name FROM information_schema.columns WHERE table_name = 'some_table'` → gives you all columns in a table.

`information_schema` can run, but it cannot hide.

2.8 GROUP_CONCAT — squashing many rows into one

UNION breaks if your two selects return different column counts. So if you ask `information_schema.tables` for all table names and there are 7 of them, you'd be returning 7

rows of 1 column. That's fine row-count-wise but you usually want to see them stacked nicely.

`GROUP_CONCAT()` glues multiple rows into a single string with a separator you choose. Example:

```
GROUP_CONCAT(table_name SEPARATOR ', ')
```

Returns `users, orders, products, sessions` as one value, in one row, in one column. UNION-friendly.

2.9 Mitigations — how the corporate world stops this

When asked in class about real-world defence, here's what we covered:

- **Input validation.** If the field is meant for an email, reject anything that isn't email-shaped. No semicolons, no dashes, no quotes.
- **Prepared statements / parameterised queries.** The query structure is sent to the database *separately* from the user input, so the input can never change the query's meaning.
- **ORMs.** Frameworks like Django (Python), Hibernate (Java), Sequelize (Node) abstract SQL away and handle parameterisation automatically. "Batteries included."
- **Active monitoring.** Go look at what people are submitting to your database. The weird stuff in your logs is your early-warning system.

2.10 Blind SQLi — a teaser for later

We didn't go deep on this, but: blind SQLi is when the page doesn't show you query output. You have to infer the answer from *behaviour* — page loads vs error, fast response vs slow response, true vs false. Same principles, harder feedback loop. Tools like **sqlmap** automate this. We'll get there.

2.11 Reality check on what SQLi gets you

Real corporate databases store passwords as **hashes**, not plaintext. So even if you dump a users table, you don't have passwords — you have hashes you'd need to crack with John the Ripper, Hashcat, Hydra, etc. We'll cover password attacks separately.

For bug bounty work, **disclosing the SQLi itself is usually enough**. You don't need to crack hashes or pivot. A clean write-up proving the injection exists is what gets paid. Stay in scope.

---

3. The lab — what to actually do

Go to: https://tryhackme.com/room/sqlinjectionlm

Work through every task in order. Capture **every flag**. Don't skip the reading sections — they'll reinforce the concepts above and add nuance on injection types (in-band, error-based, blind, out-of-band) that we only briefly touched on.

For each task, before you submit an answer:

1. Pause and predict what the underlying query looks like.
2. Decide what your payload should do.
3. Try it. If it fails, work out *why* before trying the next thing. Don't just spray payloads.
4. When it works, write down what you learnt **before moving on**. Memory fades fast — half an hour later you won't remember the small detail that made it click.

---

4. The journal — what to write

Your journal must include the following sections. Use Markdown headings. Aim for **roughly 1,500 to 2,500 words total** — long enough to be substantive, short enough to actually be read. Quality beats word count.

Required sections

4.1 Title block

Your name, student number, date of submission, lab name and link.

4.2 Introduction (≈ 150 words)

In your own words: what is SQL injection, and why does it matter? Don't define it like a textbook. Explain it like you'd explain it to a non-technical friend who asked what you're learning.

4.3 Pre-lab reflection (≈ 100 words)

What did you understand from the lesson before starting the lab? What were you unsure about? Be honest — this baseline is what you'll measure your growth against.

4.4 Walkthrough of the lab (the bulk of the journal)

Go task by task. For each significant task or level:

- **What the task asked for.**
- **The payload(s) you tried** (in code blocks).

- **Why you thought that payload would work** — your hypothesis.
- **What actually happened** — the result, the error, the response.
- **What you adjusted and why**, if your first attempt failed.
- **Screenshot evidence** of the working payload and the flag/result.

This is the section where I see your thinking. A walkthrough that just says *"I typed X and got the flag"* tells me nothing. A walkthrough that says *"I assumed the query was Y, my first attempt broke because of Z, I adjusted by doing W"* tells me you're becoming a security analyst.

4.5 Concept reinforcement (≈ 300 words)

Pick **two concepts** from the lab that took you the longest to grasp. Re-explain them in your own words, **using a fresh example that is not in the lab and not in this guide**. (E.g., apply UNION-based extraction to an imaginary e-commerce checkout, or apply auth bypass to an imaginary admin portal for a hospital system.)

This is where I'll be looking hard. If you can teach the concept in a new context, you own it.

4.6 Defensive perspective (≈ 200 words)

You've spent the lab attacking. Now flip the chair. If you were the developer of one of the vulnerable pages in the lab, **what specifically would you have done differently** to prevent the attack? Don't just say "input validation." Explain *what* you'd validate, *where* in the code, and *why* it would have stopped the specific payload you used.

4.7 Lessons learnt & next steps (≈ 150 words)

- What clicked for you that hadn't clicked before?
- What's still fuzzy?
- What would you want to learn next about SQLi or web app attacks?

4.8 References

Link to the room. Link to anything else you read or watched (PortSwigger Web Security Academy, OWASP cheat sheets, YouTube videos, etc.). Citing sources is a professional habit. Start it now.

---

5. Formatting & submission rules

- **File format:** Markdown (`.md`) preferred, PDF accepted, other wordprocessor work too.
- **Filename:** `SurnameFirstname_SQLi_Journal.md/pdf/docx` — e.g.

`MatandaTinashe_SQLi_Journal.md`.

- **Code blocks:** Use triple backticks with `sql` or `bash` syntax tags.
- **Screenshots:** Embed them inline in the Markdown using `![description](image-link-or-path)`. Crop to the relevant area — no full desktop dumps unless alternative such as word documents are used, normal insert is used.
- **Tone:** Professional, but readable. Write like a person, not a robot. First-person ("I tried…") is fine and encouraged in journals.
- **Plagiarism:** If your write-up reads like a paraphrase of the TryHackMe task descriptions or a copy of someone else's walkthrough, it'll be marked accordingly. Use your own voice. That's literally the whole point.

---

6. Marking rubric (out of 100)

Criterion	Weighting	What "excellent" looks like
Lab completion	20	All flags captured, evidenced with screenshots.
Walkthrough quality	25	Clear payloads, hypotheses stated, failures and recoveries documented, not just successes.
Conceptual understanding	20	Section 4.5 uses fresh, original examples that correctly apply UNION, auth bypass, or info_schema enumeration to a new scenario.
Defensive reasoning	10	Section 4.6 ties a specific mitigation to a specific payload, not generic advice.
Reflection & honesty	10	Pre-lab reflection and lessons-learnt show genuine self-assessment, including what's still unclear.
Structure & formatting	10	Headings, code blocks, screenshots all clean and consistent. Markdown renders without breaking.
Voice & originality	5	Reads like you wrote it. Not paraphrased lab descriptions, not AI-generated boilerplate.

Grade bands

- **80–100:** Distinction. Could pass as the first draft of a junior pentester's report.
- **65–79:** Solid. Concepts are owned, write-up is professional, minor gaps.
- **50–64:** Pass. Lab done, but walkthrough is mechanical and reflection is thin.
- **Below 50:** Resubmit. Either the lab isn't complete, the journal is copy-paste, or the conceptual section misses the mark.

7. Deadlines & support

- **Submission deadline:** *(to be confirmed in Google Classroom)*
- **Stuck on a task?** Post in the class channel. Show me what you've already tried — that's the only way I can help meaningfully. "I'm stuck" tells me nothing. *"My UNION is throwing an error and here's the payload"* tells me everything.
- **Stuck on the journal?** Same rule. Send me what you've drafted, even if it's three sentences.

---

8. Final word

This assignment isn't just about SQL injection. It's about building the two habits that separate a tier-1 analyst from a tier-3 analyst:

1. **Critical thinking under uncertainty.** You will rarely have full information. You will guess, test, fail, adjust. Get comfortable with that loop.
2. **Documenting your thinking.** A finding that isn't written down clearly may as well not exist.

Have fun with it. SQLi gets *very* satisfying once it clicks.

— Tinashe Matanda Cybersecurity Instructor